Lecture 16: System Tradeoffs: Performance, Cost, and Design Choices

AC215 / E115

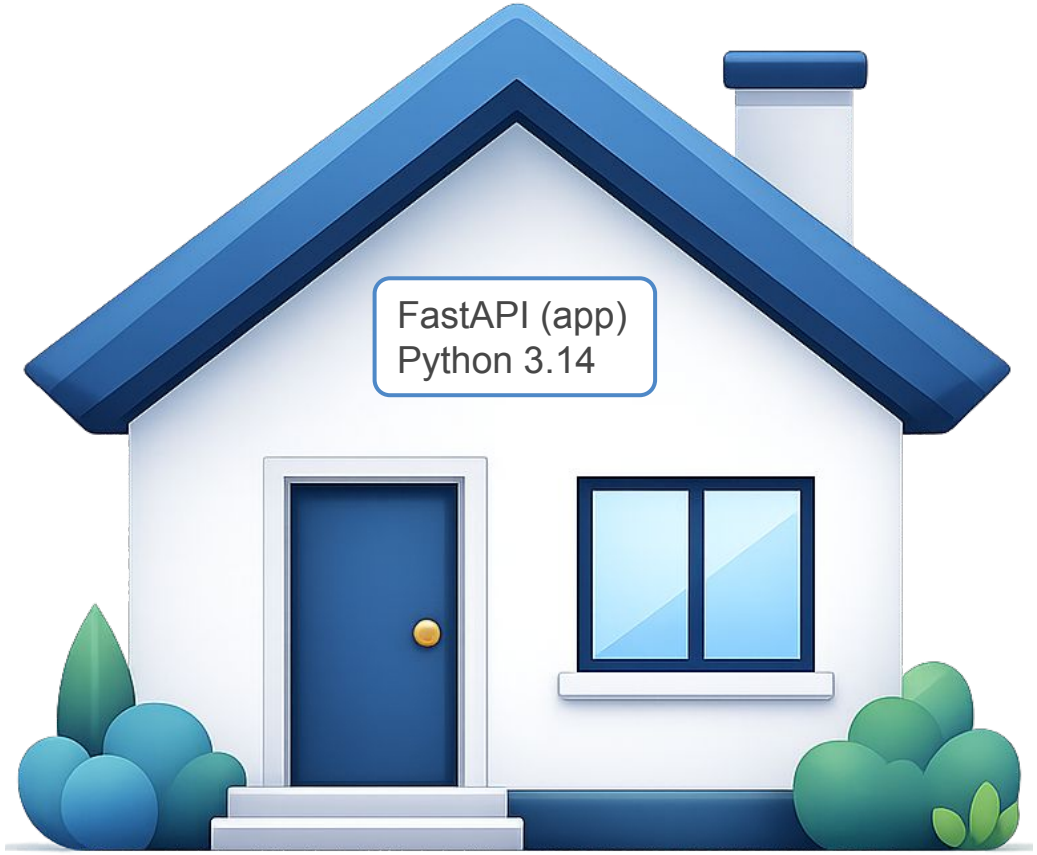
Pavlos Protopapas

SEAS / Harvard

Outline

1. **Prerequisites**
2. The Cloud Service Spectrum
3. Beyond Compute: Frontend, Databases & Services
4. What Should I Use?

Application Runtime — Local Laptop vs. Deployed Service



Deployment adds a configured network endpoint and an operating lifecycle around the application.

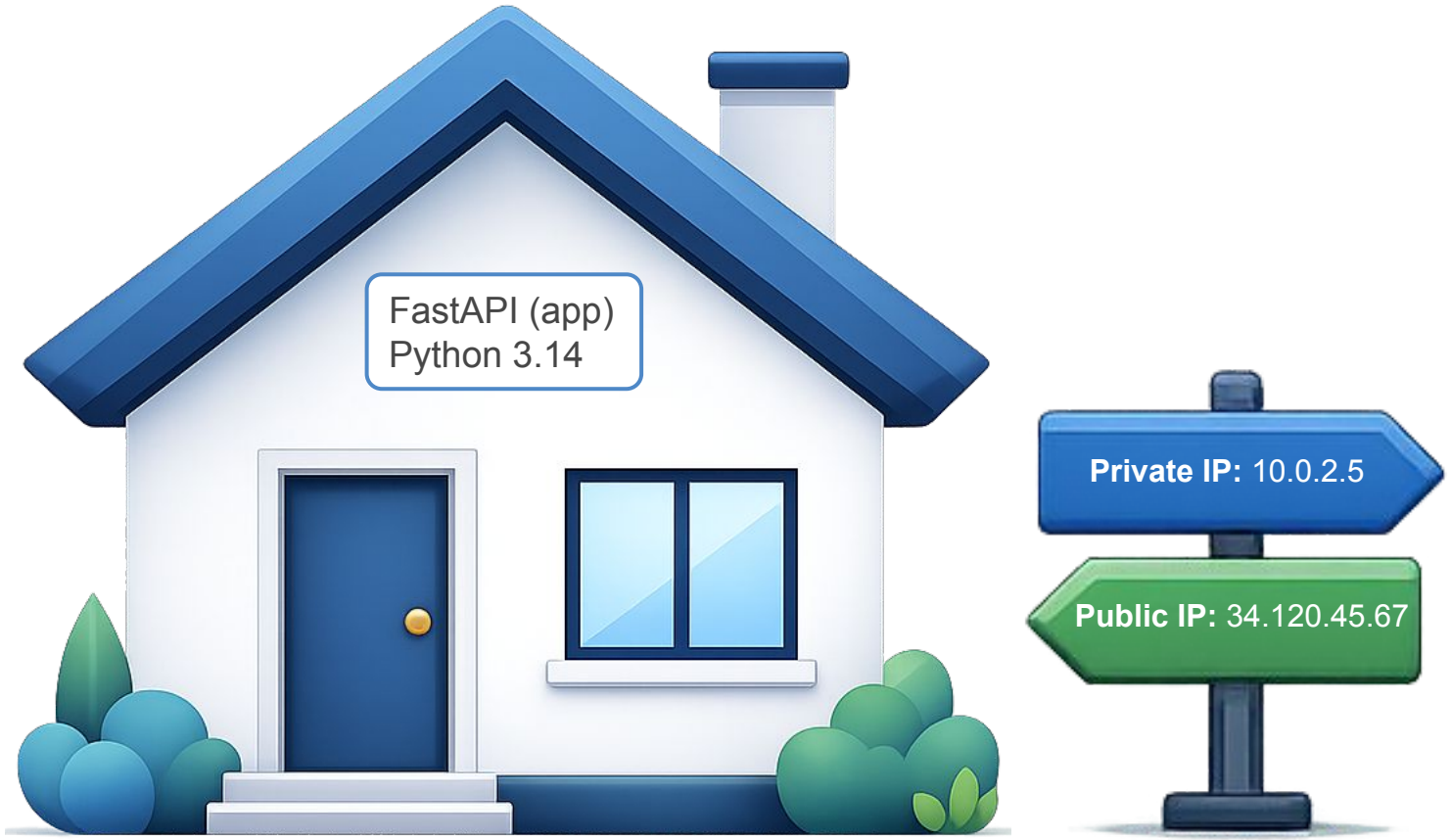
Local Laptop

- **Lifetime:** Runs while the laptop and application process are running.
- **Reachability:** Normally limited to the same device.
- **Endpoint:** `http://localhost:8000`
- **Availability:** Stops with the device or process.

Deployed Service

- **Lifetime:** Runs while the service is provisioned.
- **Reachability:** Depends on network and access policy.
- **Endpoint:** `https://app.example.com`
- **Availability:** Follows service and operating configuration.

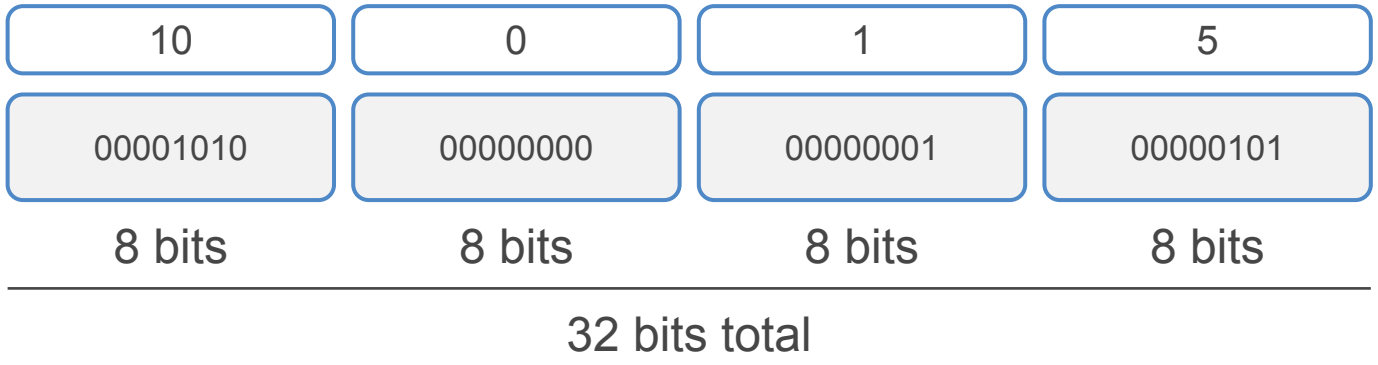
Internet Protocol (IP) Addresses



Private IPs are not publicly routed; public-IP reachability still depends on routes and firewalls.

CIDR: Classless Inter-Domain Routing

IP_ADDRESS / PREFIX_LENGTH



/32 = one IPv4 address. /0 = the entire internet.

Prefix	Fixed bits	Vary bits	Addresses
/32	32	0	1 address
/24	24	8	256
/16	16	16	65,536
/8	8	24	~16.7M
/0	0	32	entire internet

In-Class Question: CIDR

How many IPv4 addresses are in this CIDR block?

`10.0.0.0/12`

Choose one answer.

A 256

B 4,096

C 1,048,576

D Invalid CIDR

CIDR Answer — Count the Varying Bits

`10.0.0.0/12`

IPv4
32 total bits

→

PREFIX /12
12 fixed bits

→

REMAINDER
32 − 12 = 20
varying bits

→

ANSWER C
2^{20} = 1,048,576
addresses

General rule: varying bits = 32 − prefix length; address count = 2^(varying bits)

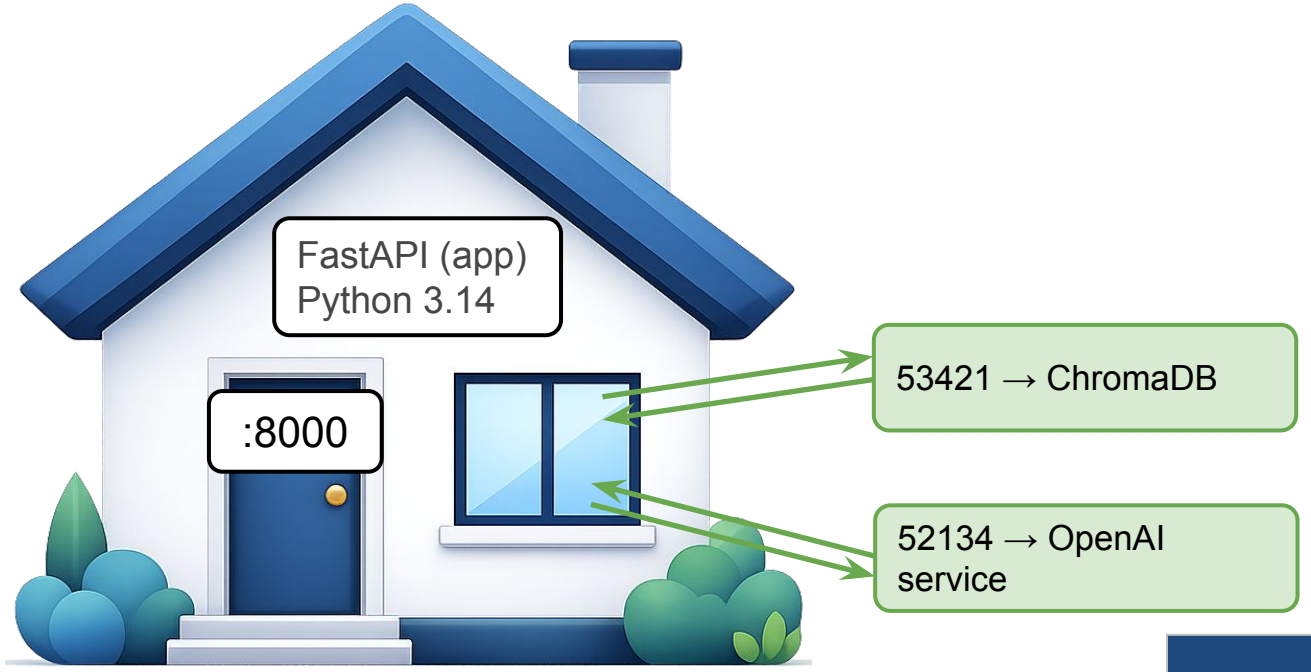
IP Address Categories

- **Loopback example —** `127.0.0.1:` Talking to yourself; the packet never leaves the operating system.
- **Bind to every local interface —** `0.0.0.0:` Listen on every local IPv4 interface.
- **Public-IP example —** `34.120.45.67:` Publicly routable; reachability still depends on routes and firewalls.

Private IPv4 ranges	Range	Use
10.0.0.0/8	10.0.0.0 - 10.255.255.255	cloud networks
172.16.0.0/12	172.16.0.0 - 172.31.255.255	Docker bridge, enterprise
192.168.0.0/16	192.168.0.0 - 192.168.255.255	home routers, small office

Request for Comments (RFC) 1918 standardizes these private IPv4 ranges.

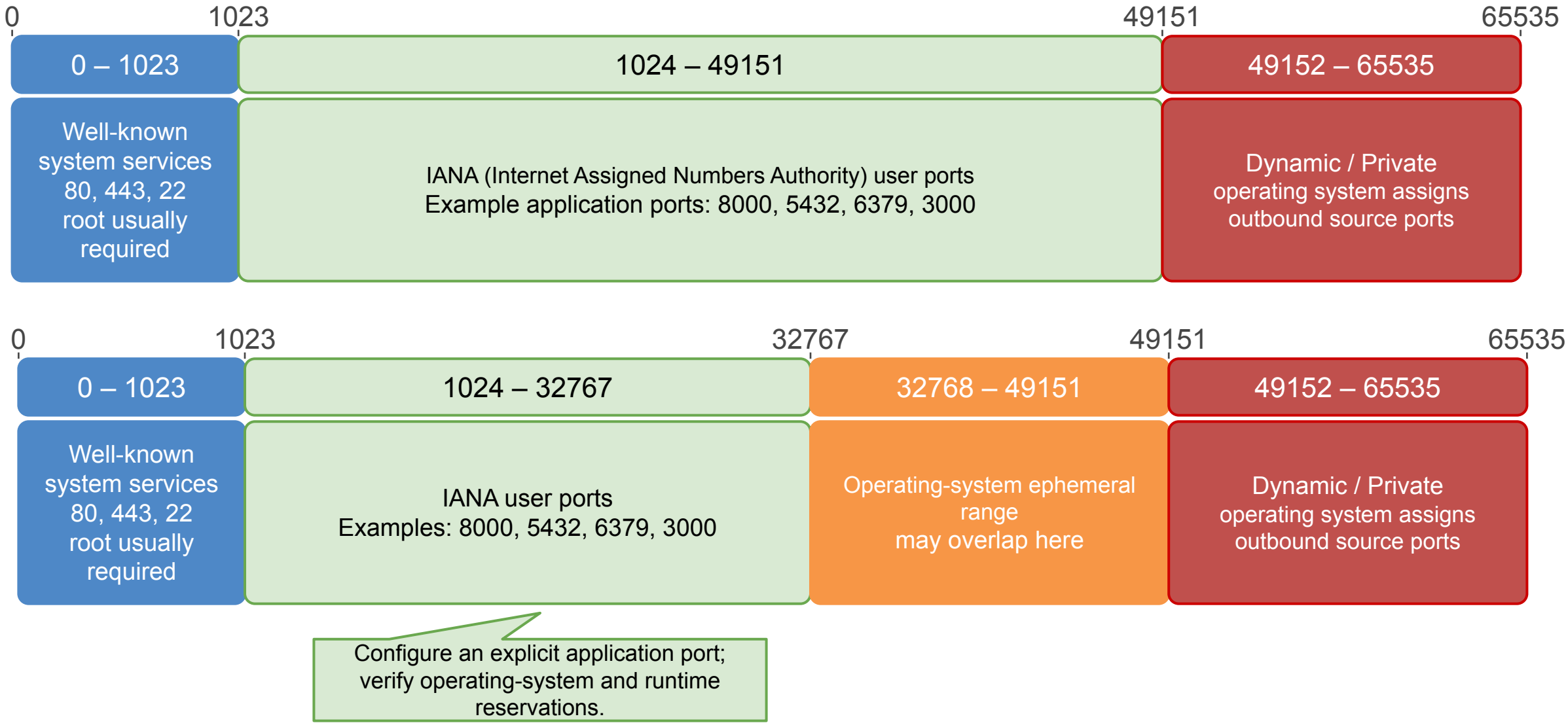
Ports — The House with Doors and Windows



Connection direction is determined by the initiator. If FastAPI calls OpenAI, the flow is outbound; a stateful firewall allows its return traffic automatically.

Direction	Port	Configured by
Inbound connection	Configured listener or mapping	Application or service
Outbound connection	Ephemeral source port	Operating system
Response	Existing connection state	Stateful firewall

Port Number Ranges



Security Groups — Who Can Knock on the Door?

A port identifies the door. The security group evaluates an allow list.

PORT



+

ALLOW RULE



Transmission Control Protocol (TCP)
destination port: 8000
allowed source: Nginx

ALLOW rules only
No explicit DENY

=

GUARDED PORT



MATCH: ALLOW | NO MATCH: BLOCK

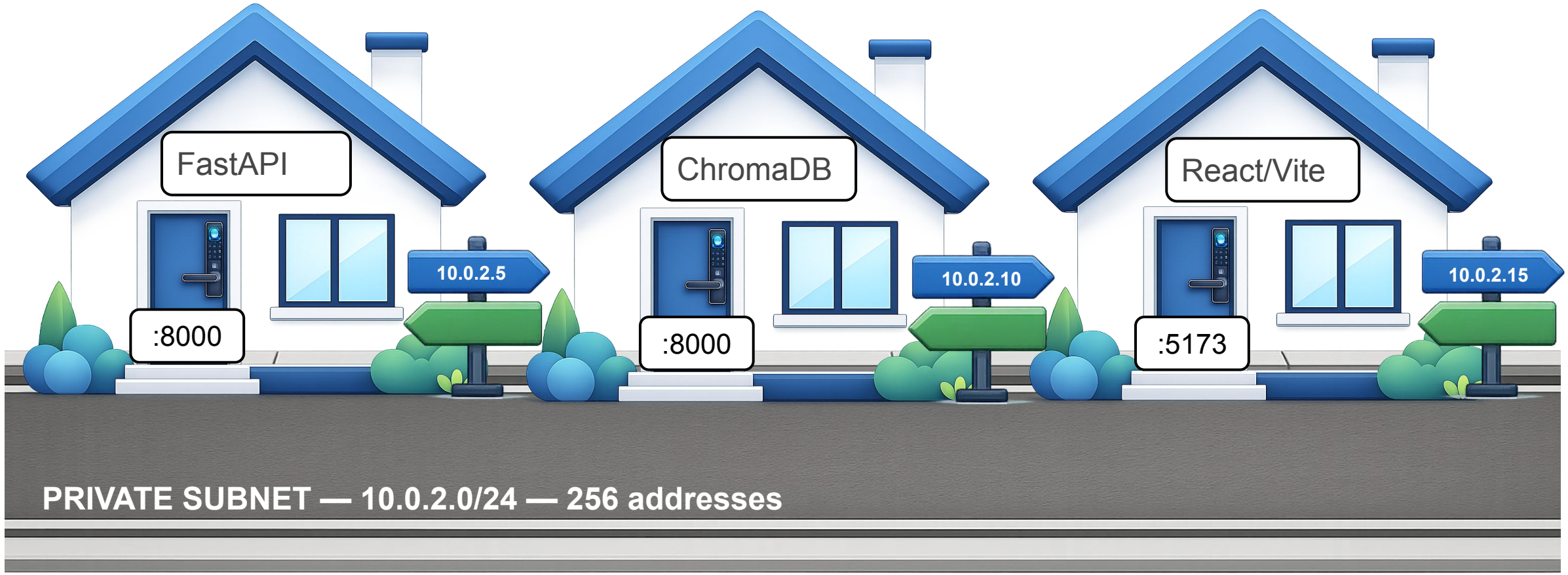
INBOUND
Starts with no rules.
Add a matching allow.

OUTBOUND
Starts allow-all.
Narrow to matching rules.

STATEFUL
Responses to allowed connections return automatically.

Subnets — Grouping IPs Into Streets

A subnet is a group of IP addresses that belong together — like a street in a neighborhood.

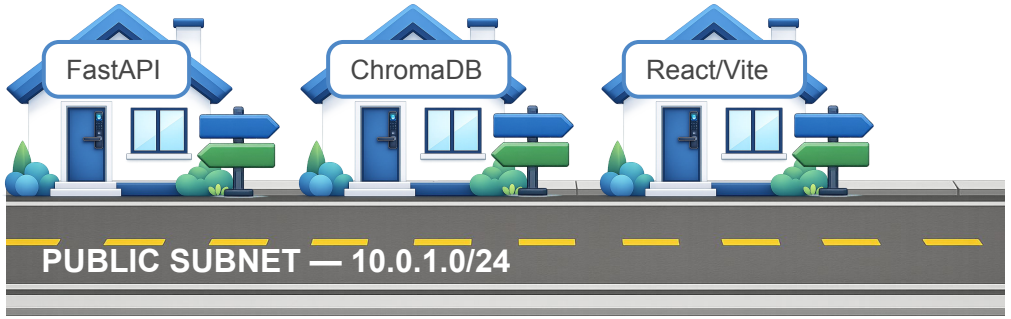


Subnet = a street where houses (servers) live. Security group = the FaceID door lock on each house's door.

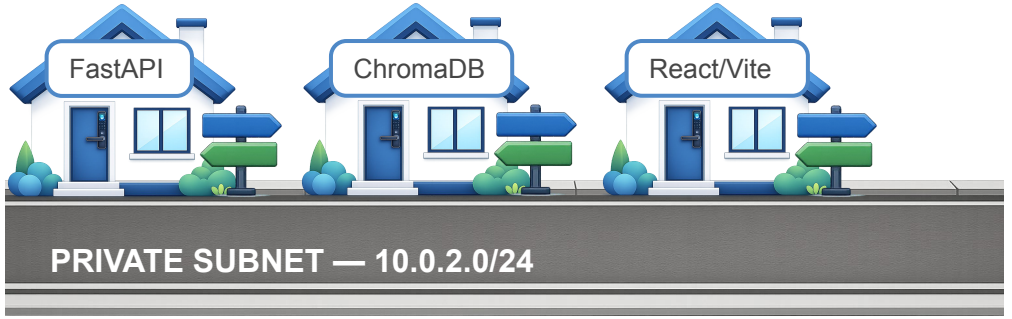
Public vs. Private Subnets



PUBLIC SUBNET
Direct route to an Internet Gateway



PRIVATE SUBNET
No direct route to an Internet Gateway



Public subnet route table

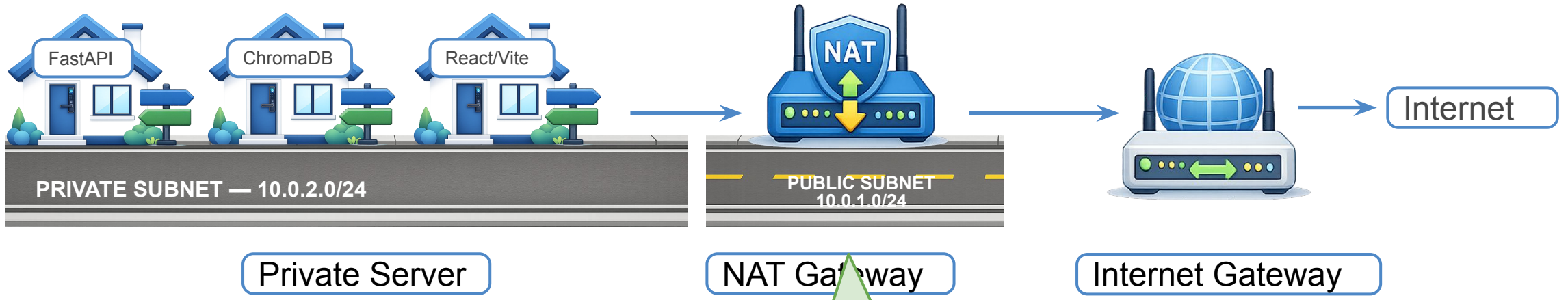
Destination	Gateway
10.0.0.0/16	local route
0.0.0.0/0	Internet Gateway

Private subnet route table

Destination	Gateway
10.0.0.0/16	local route
0.0.0.0/0	NAT Gateway

A public subnet route is necessary, not sufficient: public-IP mapping and firewall rules also control reachability.

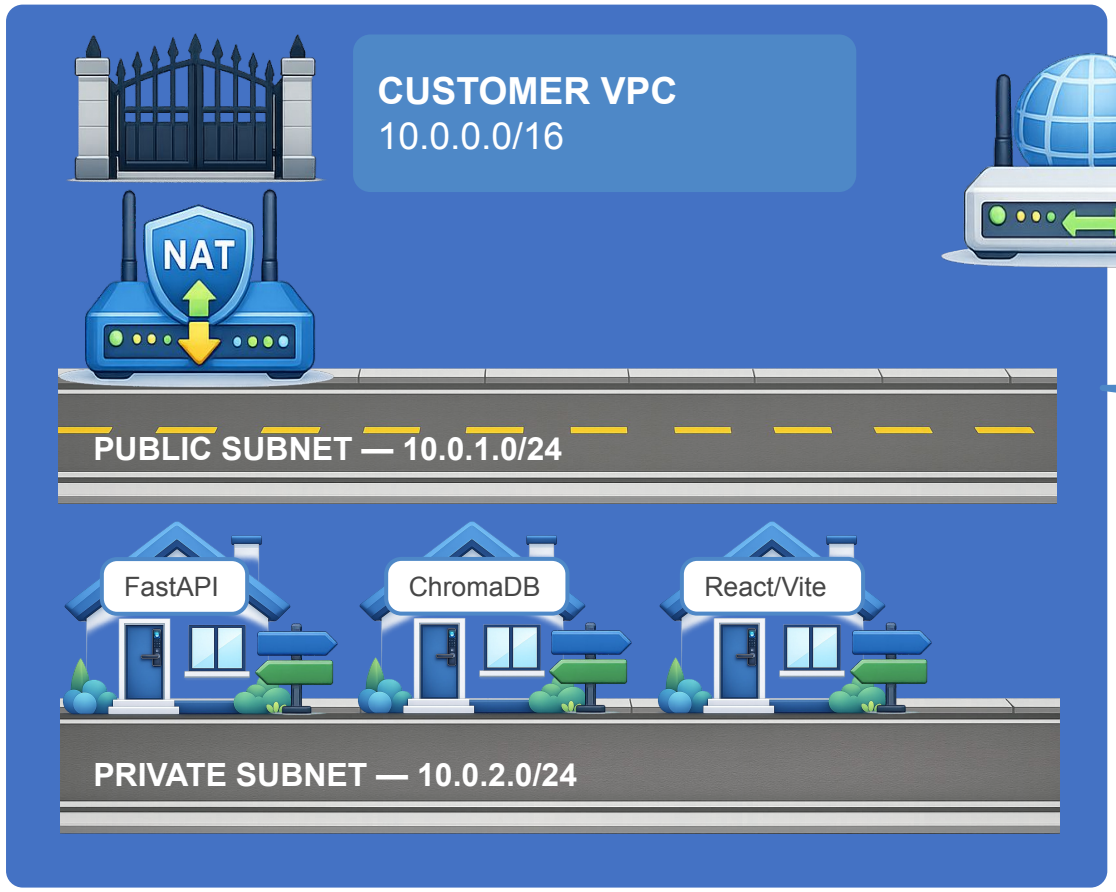
Network Address Translation (NAT) Gateway



Destination	Gateway
10.0.0.0/16	local route
0.0.0.0/0	NAT Gateway

NAT translates the private source address, remembers the mapping, and maps reply traffic back. It does not accept unsolicited inbound connections.

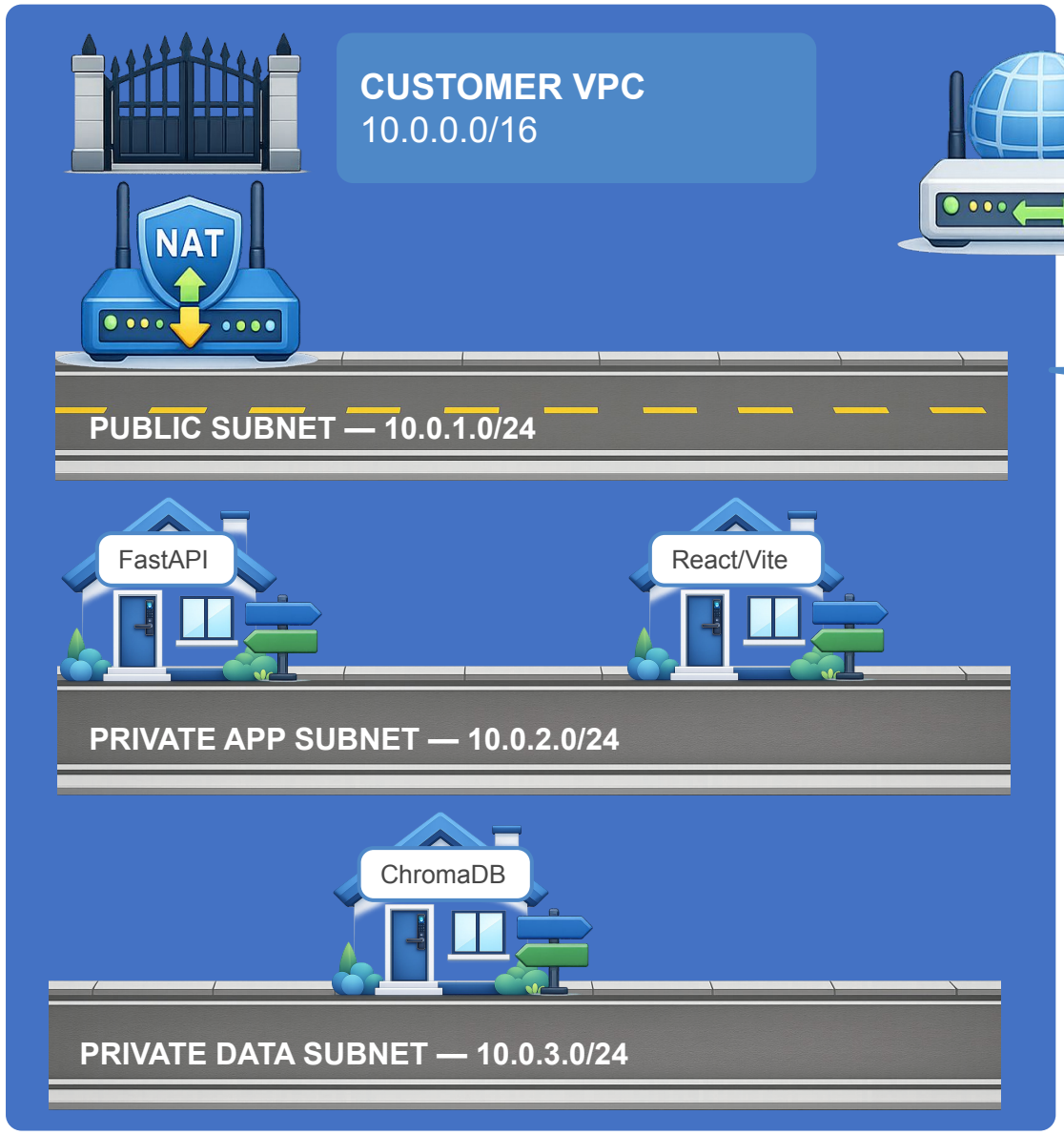
Virtual Private Cloud (VPC) — The Gated Community



Internet

Example: the /16 VPC is divided into smaller /24 subnet ranges.

VPC Diagram — The Gated Community

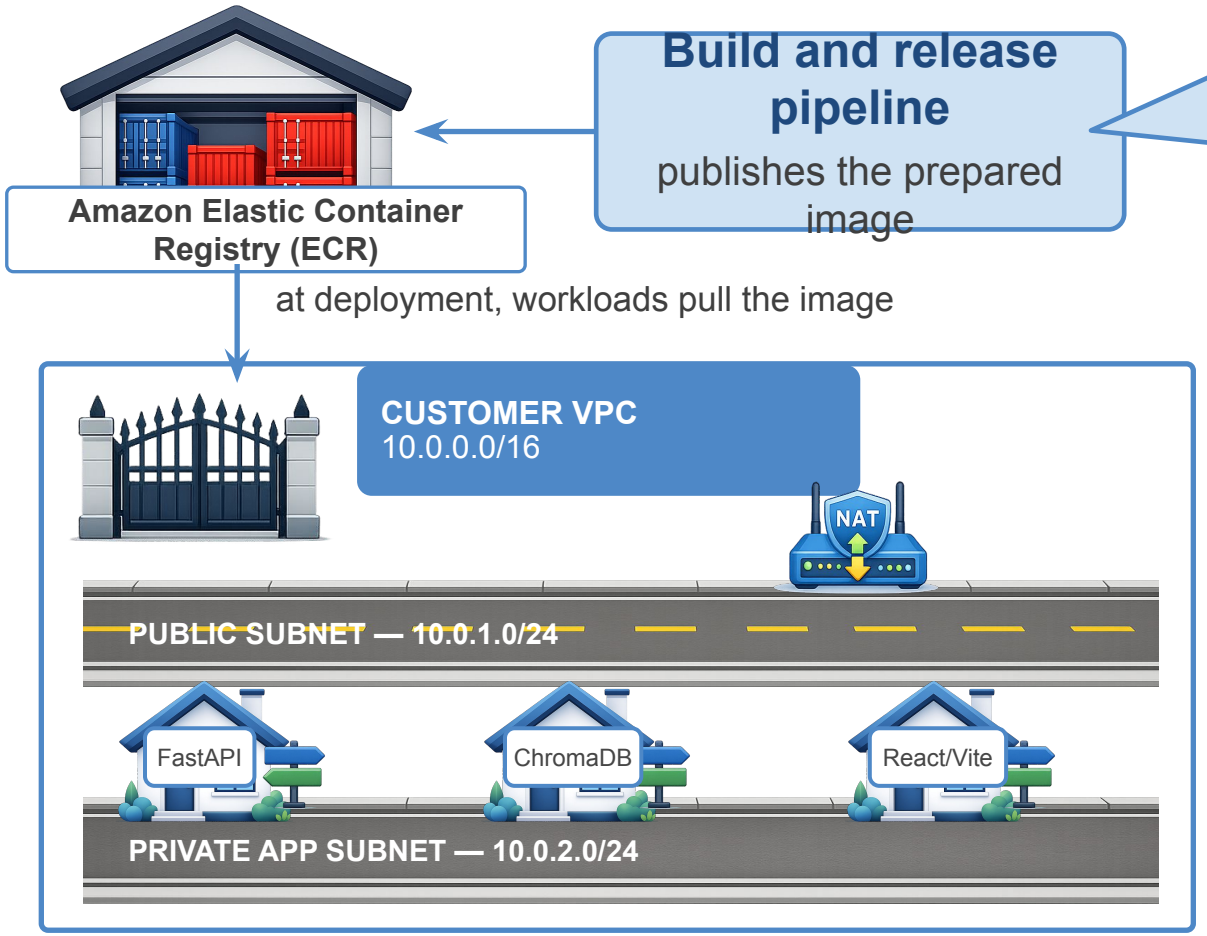




Internet

Example: the /16 VPC is divided into three /24 subnet ranges.

Docker in the Cloud — Container Registry



Before deployment, the build pipeline prepares one image

It packages the app as a container image, checks that it works, scans for known vulnerabilities, and gives it a version label (tag).

The registry is the deployment handoff

The registry stores the approved image. At deployment, workloads pull that exact version—no rebuild.

Keep the image close to the workload

Use the same Region by default; replicate the image for multi-Region deployments.

Same Container, Different Host



REGISTRY →

Same approved image — myapp:v1.4

Same FastAPI listener — 0.0.0.0:8000

LAPTOP

Docker host

```
docker run -p 8080:8000 myapp:v1.4
```

LAPTOP NETWORK (HOST)



localhost:8080

FastAPI container

published endpoint on the laptop

AMAZON EC2

cloud virtual machine on a remote Linux host

Docker maps VM :8000 to container :8000

CUSTOMER VPC — 10.0.0.0/16



10.0.2.5:8000

PRIVATE APP SUBNET — 10.0.2.0/24

VPC routes and security rules govern access to the VM endpoint.

Next: trace the laptop port mapping, then add a public cloud endpoint.

Port Publishing — Laptop Endpoint to Application Listener

`docker run -p 8080:8000 myapp:v1.4` maps laptop port 8080 to container port 8000.

LAPTOP NETWORK (HOST)

Browser on the laptop

`localhost:8080`

connects to the published endpoint

Docker port publishing

`laptop :8080 → container :8000`

forwards traffic into the container

CONTAINER NETWORK

separate from the laptop network



FastAPI

listens on 0.0.0.0:8000

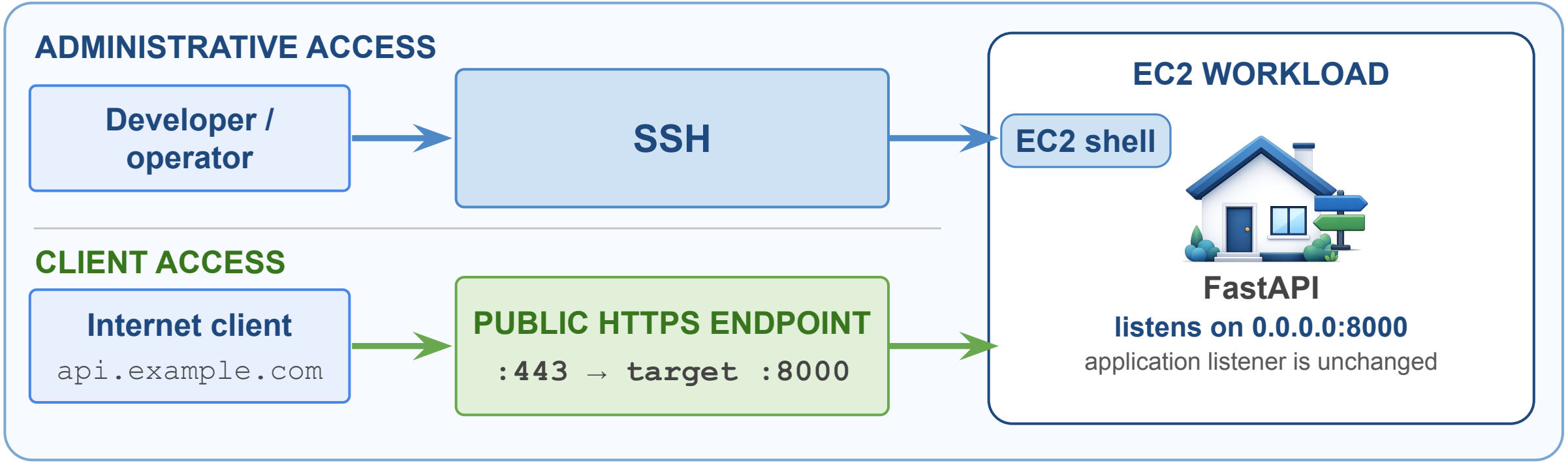
accepts traffic from the container network

The laptop endpoint and application listener are configured independently.

Their port numbers do not need to match.

Cloud Access — Administrative vs. Client Access

The EC2 workload stays fixed; the access purpose and entry path change.



Administrative access reaches the machine; client access reaches the service.

Runtime Configuration — Configure at Startup



ONE IMMUTABLE IMAGE

DEVELOPMENT



WORKLOAD

RUNTIME VALUES

```
APP_ENV=dev
LOG_LEVEL=debug
FEATURE_FLAG=true
```

PRODUCTION



WORKLOAD

RUNTIME VALUES

```
APP_ENV=prod
LOG_LEVEL=info
FEATURE_FLAG=false
```

Secrets — Inject, Don’t Bake



Secret manager

authorized delivery



workload identity

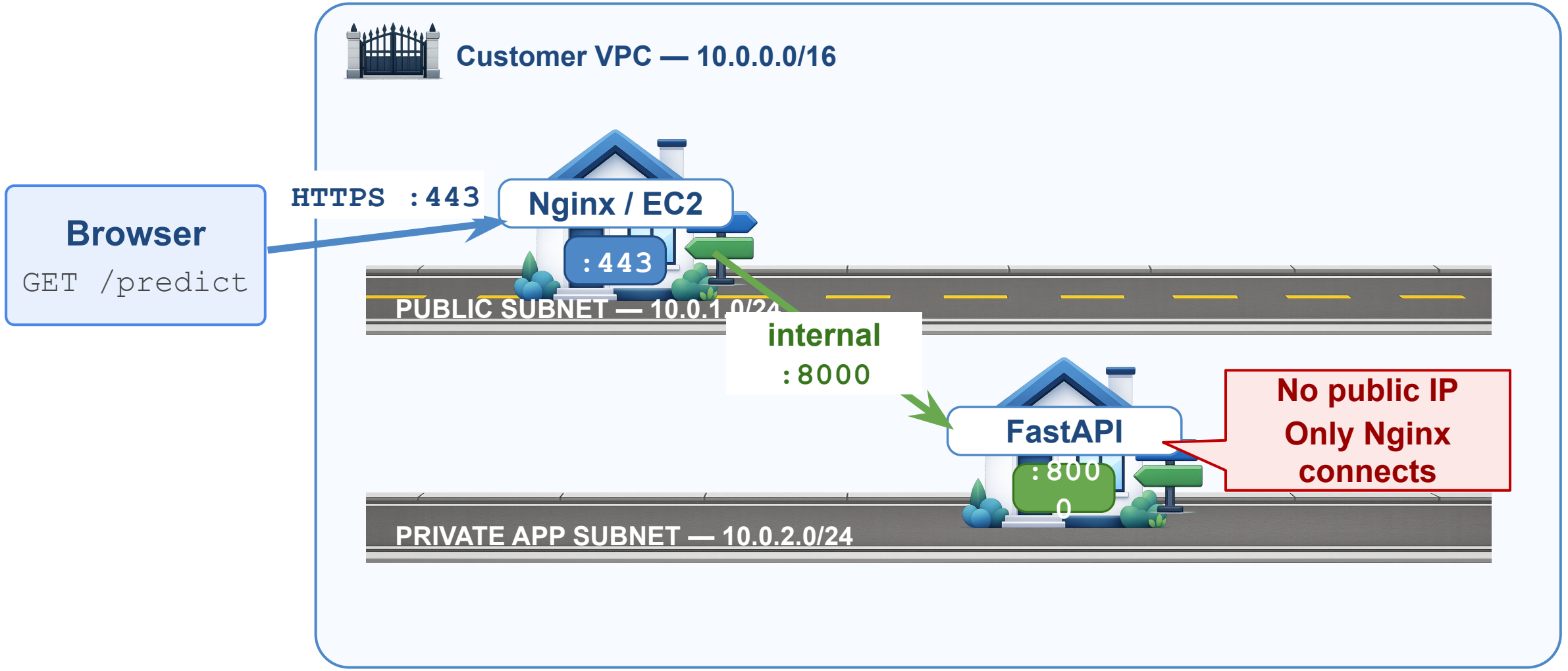
APPLICATION READS AT STARTUP

```
os.getenv("OPENAI_API_KEY")
```

Never place secrets in: source code, container images, or Git

Grant only the secrets this workload needs.

Nginx on EC2 — Keep FastAPI :8000 Private



Nginx on EC2 vs. ALB — What Moves to AWS?

NGINX ON EC2



TEAM OPERATES
guest OS + Nginx
proxy instances + scaling

TEAM CONFIGURES THE ROUTE

```
location /api/ {
  proxy_pass
http://10.0.2.5:8000;
}
```

APPLICATION LOAD BALANCER (ALB)



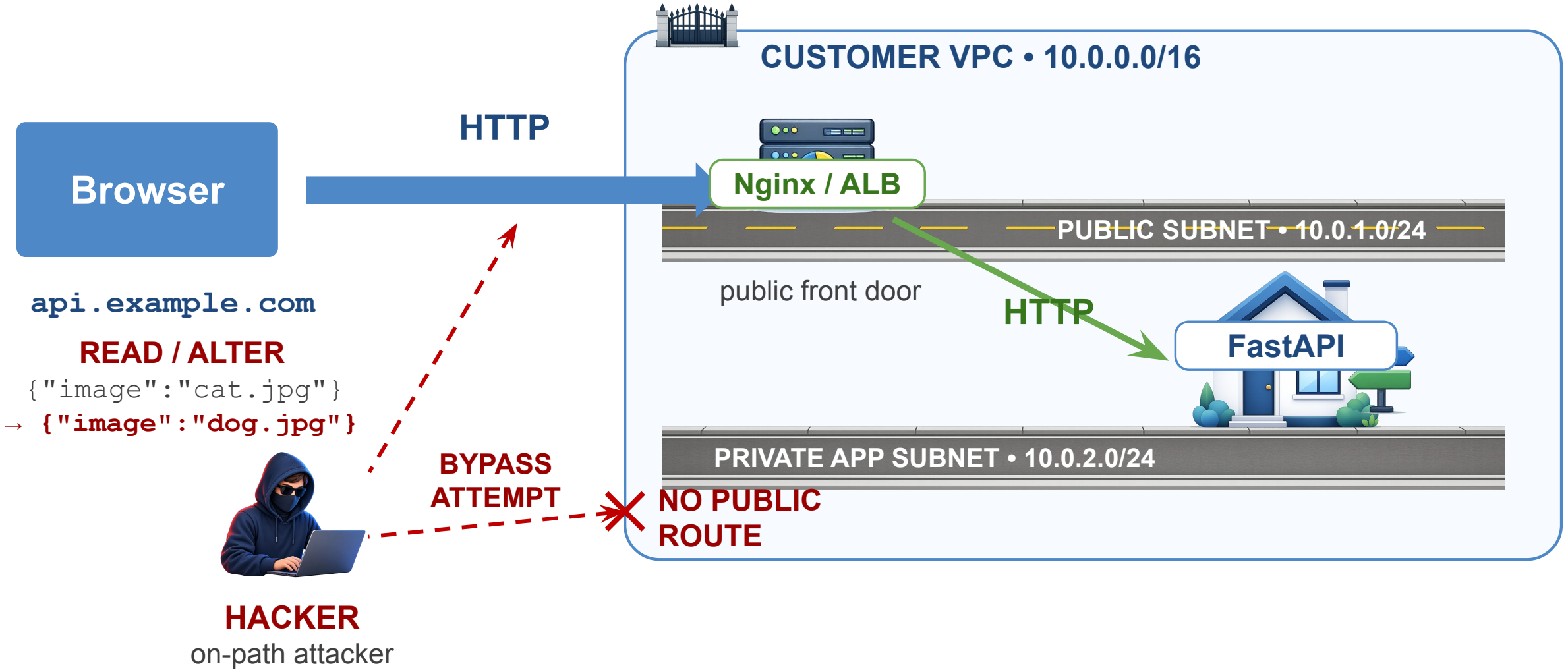
AWS OPERATES
ALB nodes + replacement
service capacity + scaling

TEAM CONFIGURES THE ROUTE

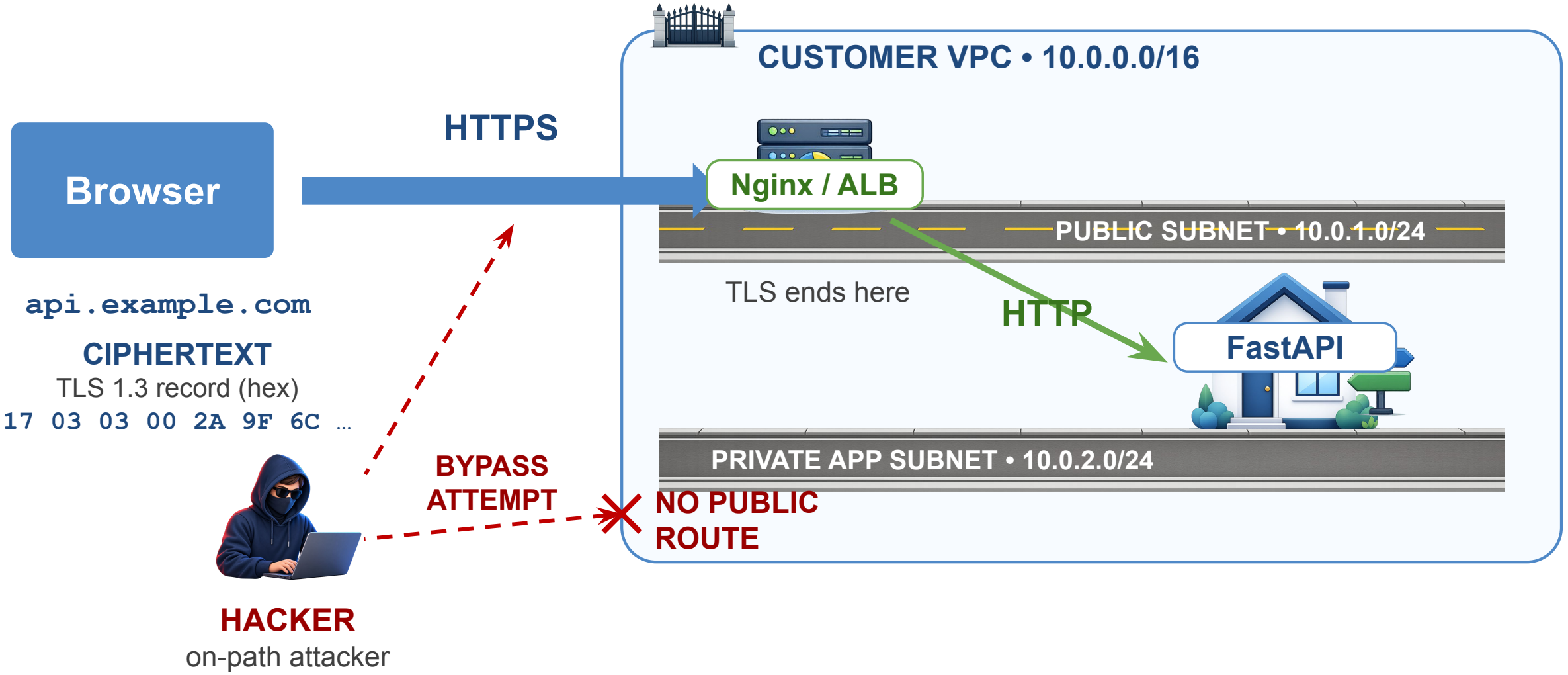
```
listener :443
path /api/* -> fastapi-tg
fastapi-tg -> targets :8000
```

Same customer-defined route; only proxy-host operations move to AWS.

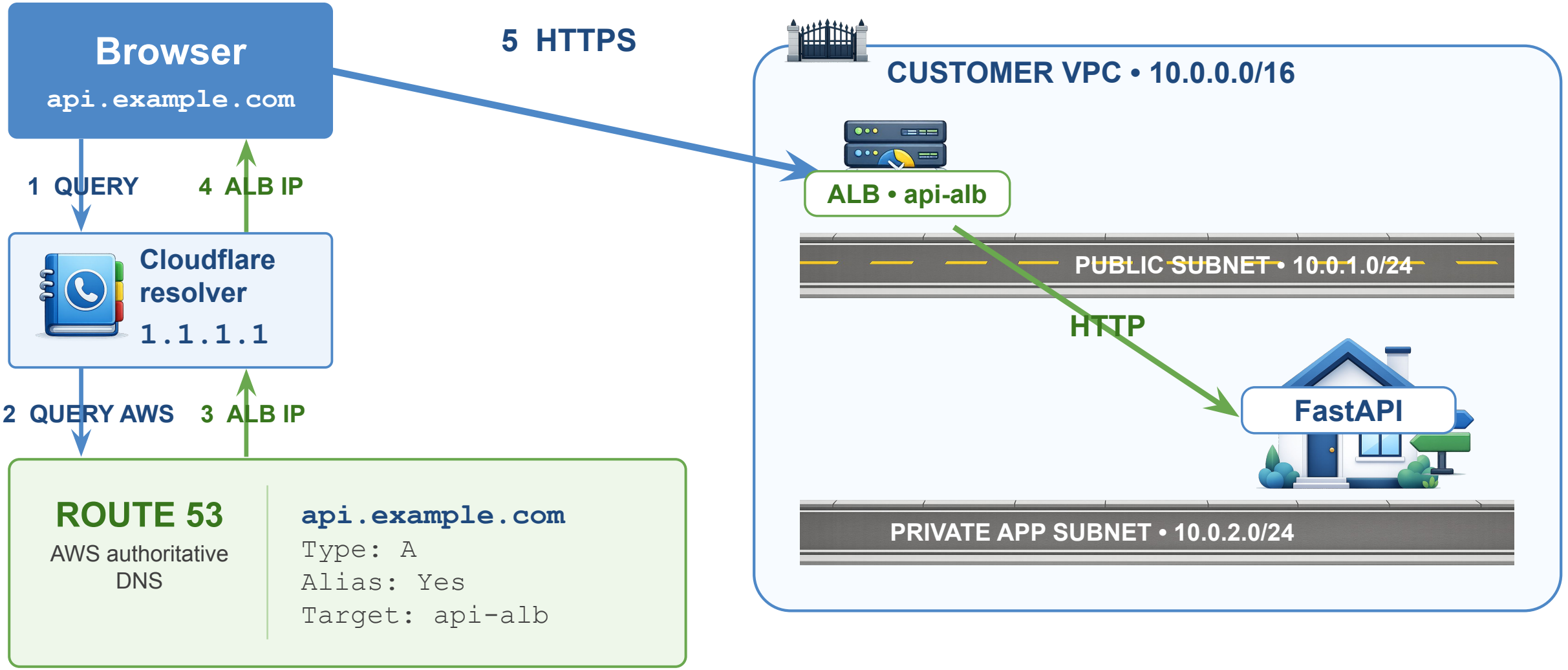
HTTP — Readable on the Public Path



HTTPS — Encrypt the Public Path



DNS — Ask the Resolver, Then Connect to the ALB



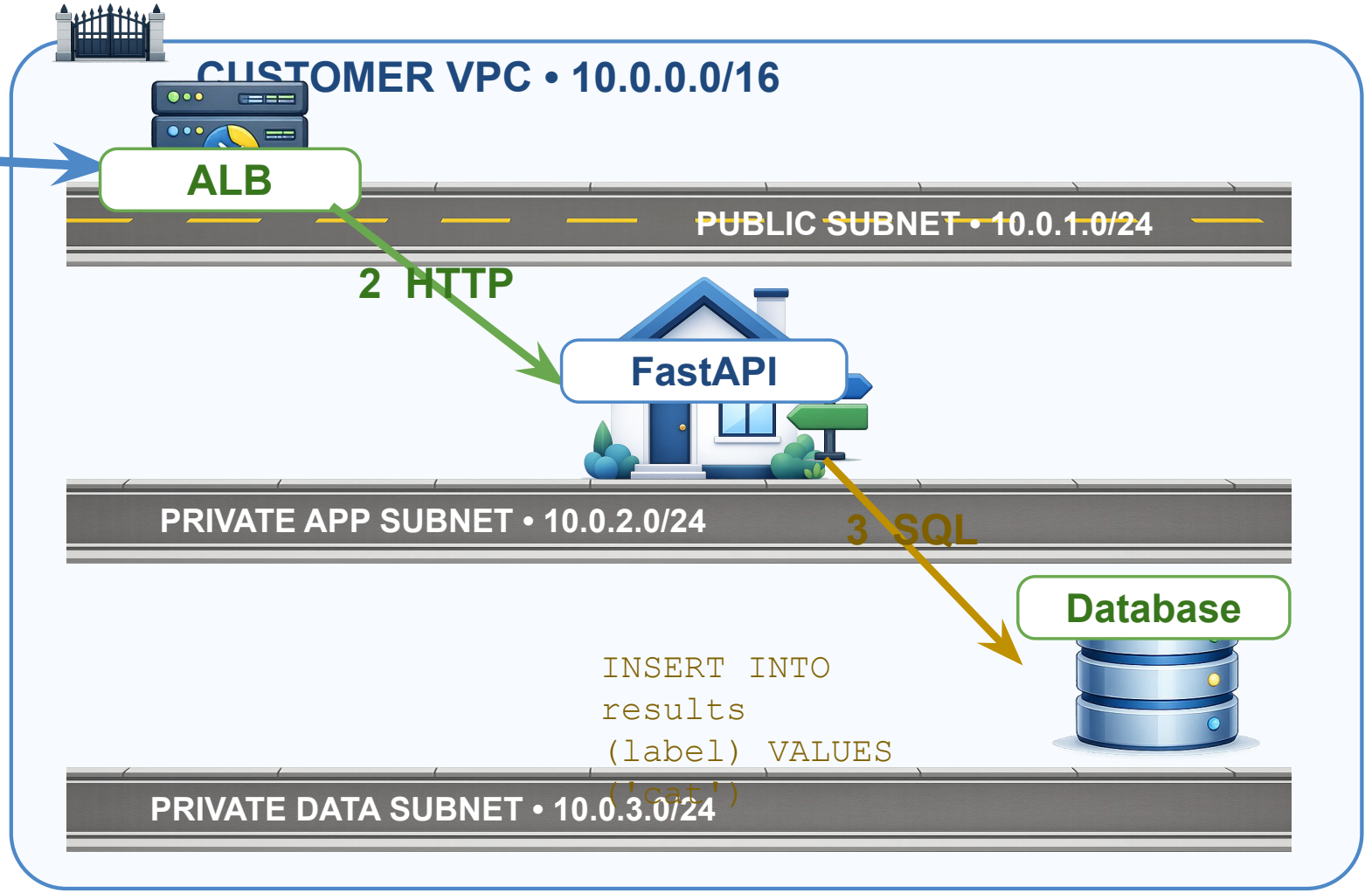
One /predict Request — Three Contracts

Browser

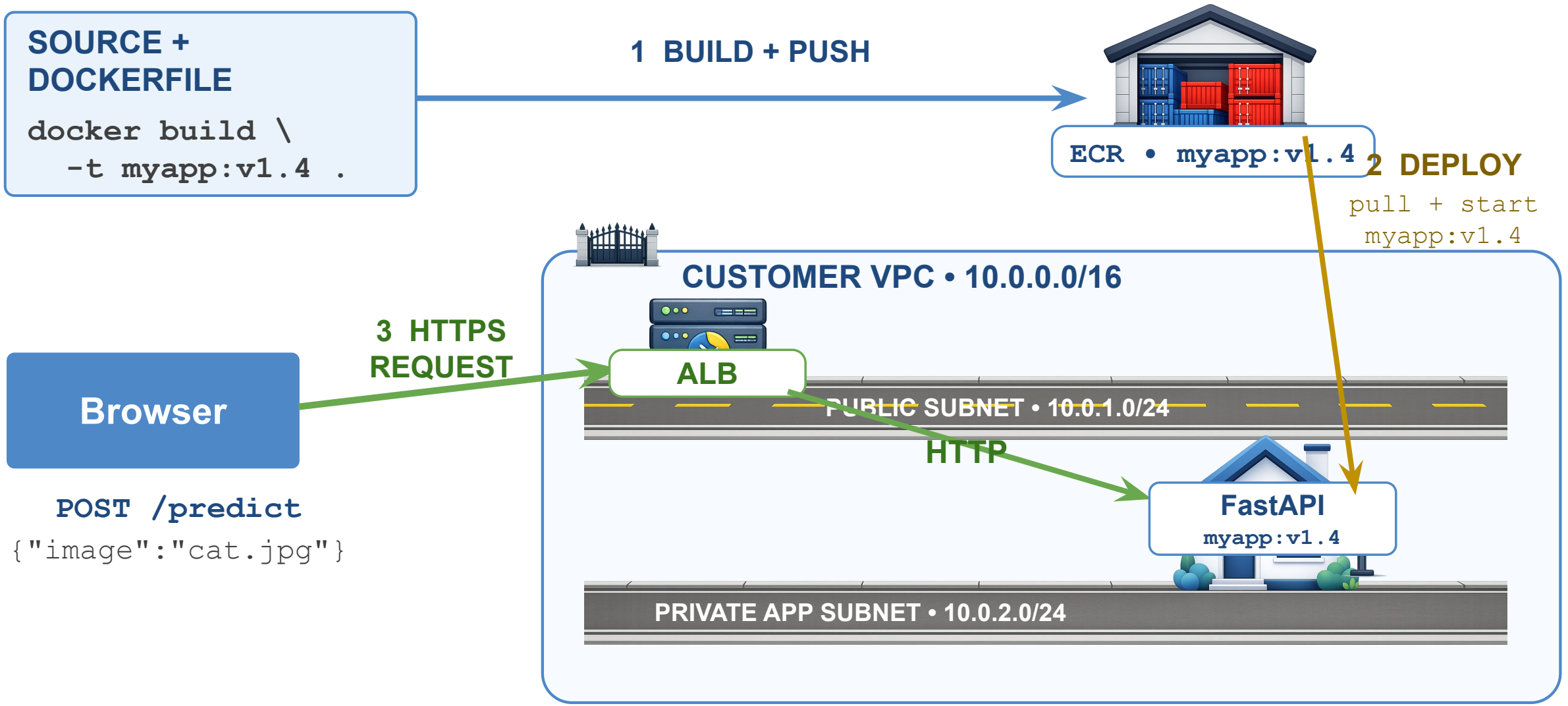
1 HTTPS

POST /predict

```
{"image":"cat.jpg"}
```



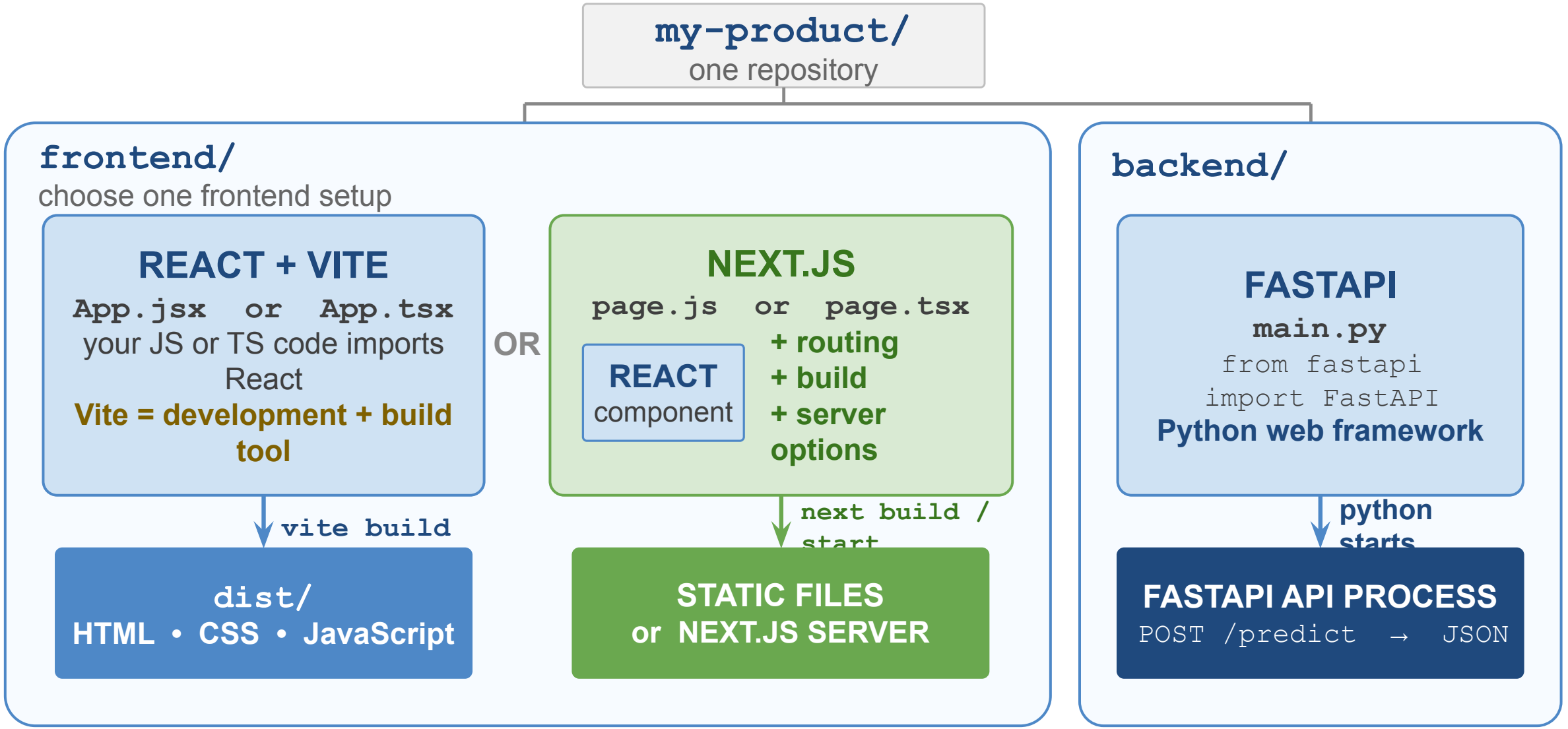
Build Once; Runtime Starts That Exact Image



The Page Loads, Then It Calls FastAPI



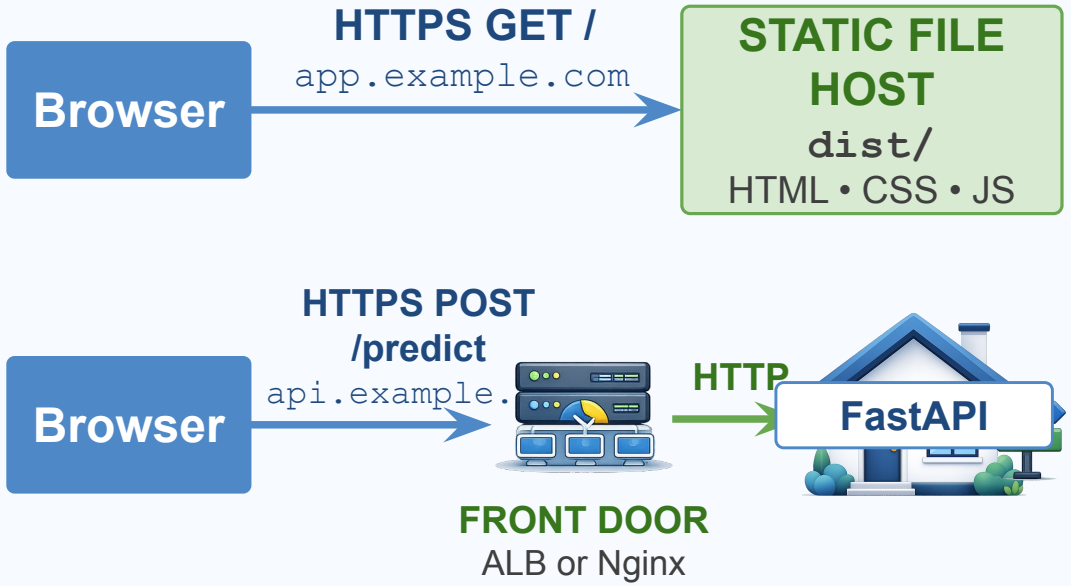
What Lives in One Web Project?



One Product, Two Common Runtime Layouts

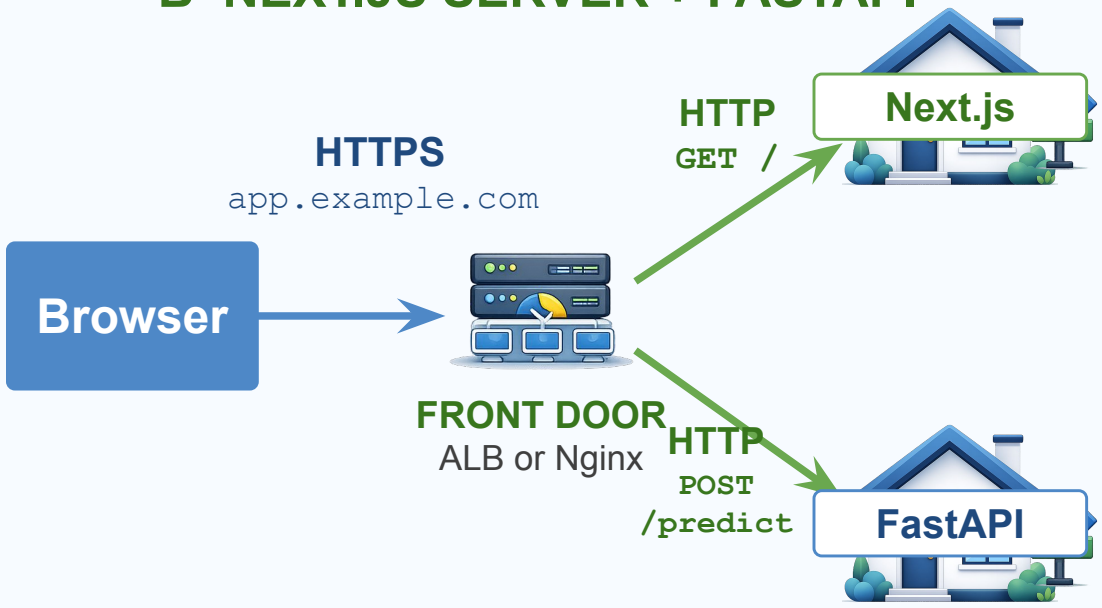
ONE PRODUCT • OFTEN ONE REPO • TWO DEPLOYMENTS

A REACT + VITE STATIC FRONTEND



SAME BROWSER • 2 HOSTNAMES • 2 DEPLOYMENTS

B NEXT.JS SERVER + FASTAPI



ONE HOSTNAME • PATH RULES • 2 DEPLOYMENTS

Outline

1. Prerequisites
2. **The Cloud Service Spectrum**
3. Beyond Compute: Frontend, Databases & Services
4. What Should I Use?

Compute Supplies Capacity; Orchestrators Place Workloads

1 COMPUTE CAPACITY

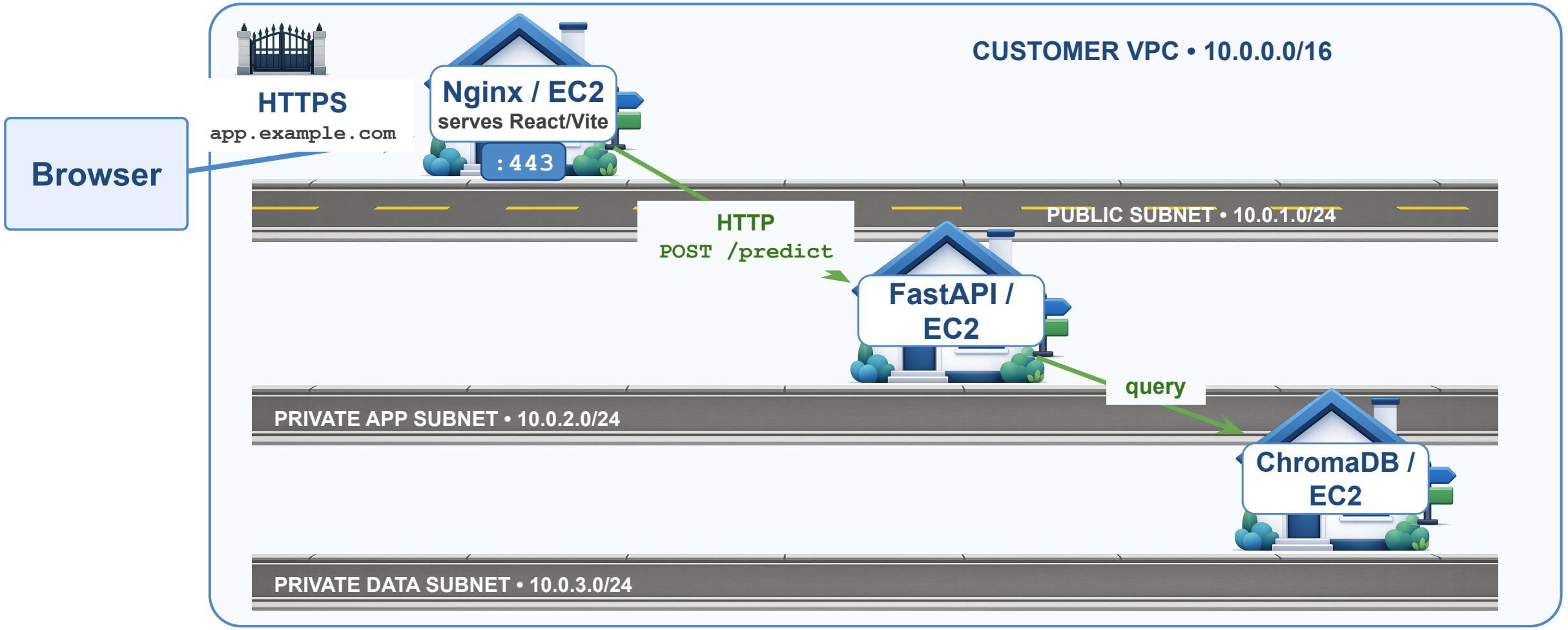
Question	EC2	Fargate
Request	VM: CPU + memory + GPU option	Task / pod: CPU + memory
Access	SSH / Session Manager	No host login
Patching	Team patches / replaces guest OS	AWS patches host OS
Capacity	Team adds / removes VMs	AWS supplies per task / pod

2 ORCHESTRATION

Question	ECS	Kubernetes
Model	AWS-native service	Open-source system
Unit	ECS task	Pod
Recovery	Keeps desired task count	Replaces failed Pods
Scaling	Auto Scaling if configured	HPA if configured

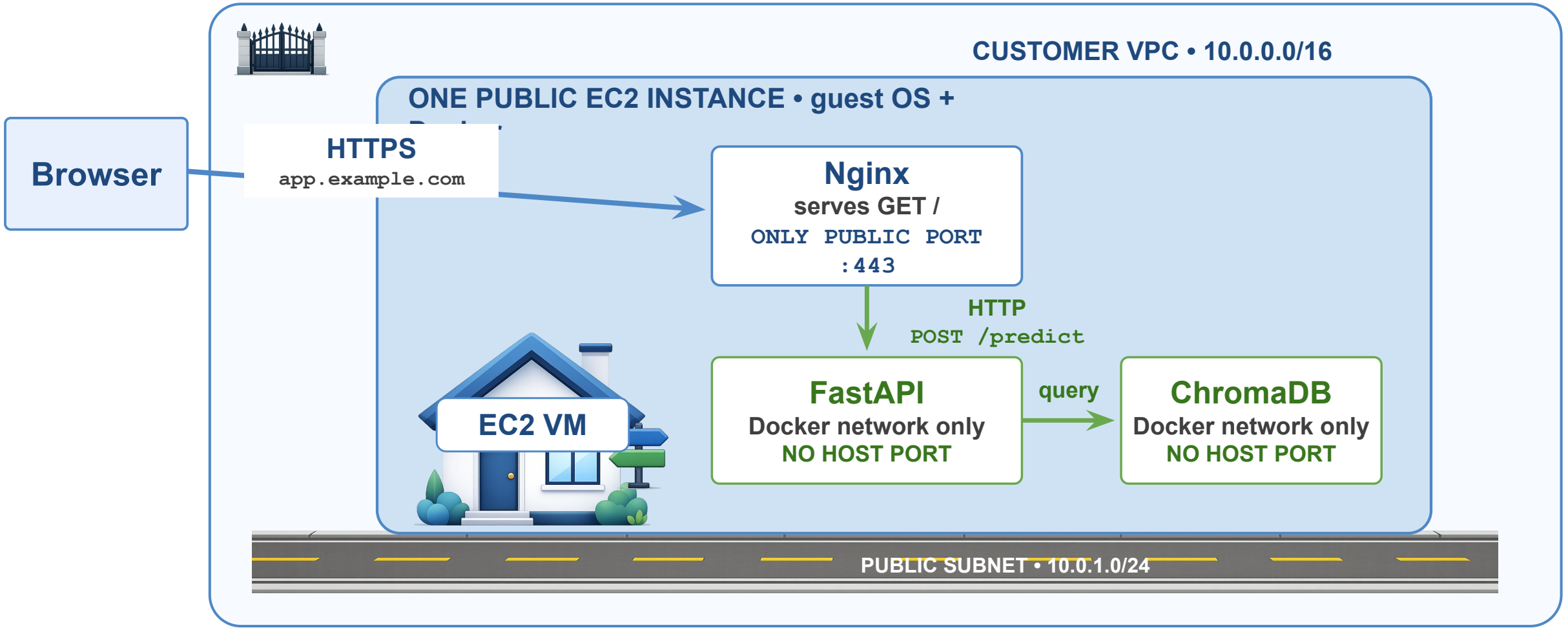
EKS = AWS managed Kubernetes
GKE = Google Cloud managed Kubernetes

EC2 — One VM per Service



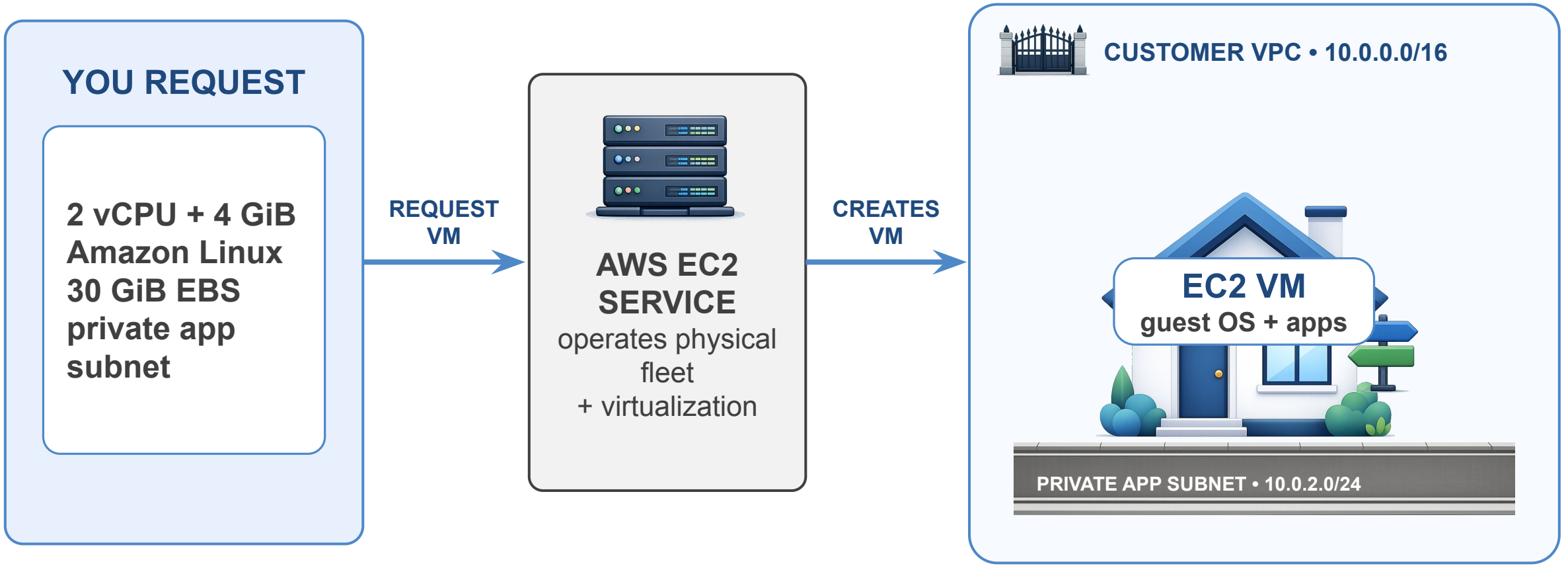
Scale or reboot one service alone • patch and monitor three guest OSs

EC2 Prototype — Several Services on One VM



One reboot stops all three • scale one service means scaling the whole VM

EC2 — Instance Contract, Not Hardware



AWS repairs hardware + virtualization • team patches the guest OS + applications

Fargate Is Capacity; ECS or EKS Does the Scheduling

YOU DEFINE

Representative task request

```
image: api:v1.4
cpu: 0.5 vCPU
memory: 1 GiB
network: private
role: api-role
```

SUBMIT →



ECS OR EKS schedules

PLACE →

FARGATE CAPACITY



1 task / pod = 1 isolated room

Fargate — Host Operations vs. Workload Configuration

AWS OPERATES



- physical hosts + hypervisor
- host OS + infrastructure patching
- isolated task capacity

TEAM CONFIGURES + OWNS



- image, command, CPU + memory
- VPC, security groups, IAM + secrets
- health/scaling, logs, data + recovery

EC2 vs Fargate — Compare the Same Operational Questions

Same question	EC2	Fargate
Host access	Guest OS via SSH / Session Manager	No host login
OS patching	Team patches or replaces guest OS	AWS patches Fargate host OS
Capacity	Team launches or terminates VMs	AWS supplies capacity per task / pod
Isolation	Containers share a VM boundary	Each task / pod gets isolated capacity
Best fit	Need guest-OS or host control	Prefer no guest-OS administration

Task and Pod — The Scheduling Wrapper

ECS TASK




containers move, restart, and scale together

KUBERNETES POD




containers share a lifecycle and network namespace

Same Wrapper, Different Capacity

Task / Pod
workload wrapper

capacity contract



EC2 capacity
team patches and scales the VMs



Fargate capacity
AWS provisions isolated task capacity

Same workload wrapper; different host operator.

ECS — Reconcile Desired State

TASK DEFINITION

`api:v1.4`

DESIRED COUNT = 3



AWS-OPERATED ECS CONTROL PLANE

TASK 1

`api:v1.4`

TASK 2

`api:v1.4`

TASK 3

`api:v1.4`

ECS starts a replacement task

ECS: What, How Many, and Where



WHAT

TASK DEFINITION

`image: api:v1.4`

CPU + memory + ports

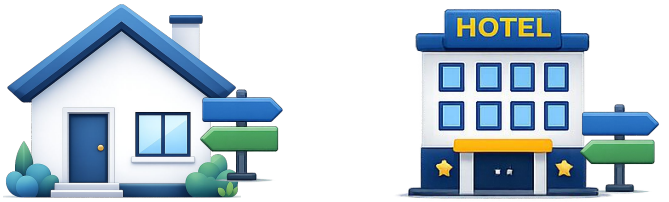


HOW MANY

ECS SERVICE

`desired count: 3`

maintains three tasks



WHERE

EC2 capacity **Fargate capacity**

LAUNCH TYPE / CAPACITY PROVIDER

ECS Scaling — Workload Count vs Capacity

WORKLOAD SCALING

desired tasks

3 → 6

controller changes workload count

CAPACITY SCALING



1

enough places for requested tasks

Work can remain pending when capacity lags.

ECS Networking — AWS Control Plane Outside Your VPC



AWS-operated ECS control plane



customer VPC
10.0.0.0/16





routing + security

PRIVATE WORKLOAD SUBNET — 10.0.2.0/24

EC2 CAPACITY

FARGATE CAPACITY

EKS Control Plane Is Outside Your VPC



AWS runs Kubernetes API servers + etcd









customer VPC
10.0.0.0/16

PRIVATE WORKLOAD SUBNET — 10.0.2.0/24

node + pods | node + pods | node + pods

Your team still configures worker capacity, networking, identity, policy, and workloads.

EKS — What AWS Runs vs. What Your Team Configures

AWS RUNS FOR EKS



- Kubernetes API servers
- etcd + control-plane hosts
- control-plane patching + availability

TEAM CONFIGURES + OPERATES



- EC2 nodes or Fargate profiles
- add-ons, VPC, IAM + policy
- upgrades, observability, workloads + data

ECS or EKS — Operating Contract

CHOOSE ECS WHEN



- task/service API is sufficient
- AWS runs the control plane
- team configures tasks, VPC, IAM + scaling

CHOOSE EKS WHEN



- Kubernetes API/ecosystem is required
- team chooses nodes or Fargate profiles
- team operates add-ons, policy + upgrades

Default to ECS unless a concrete Kubernetes requirement exists.

Cloud Run — Google Runs Routing, Revisions, and Instances

SEPARATE LAYERS







compute **scheduler** **capacity**

EC2 / ECS / Fargate

CLOUD RUN SERVICE



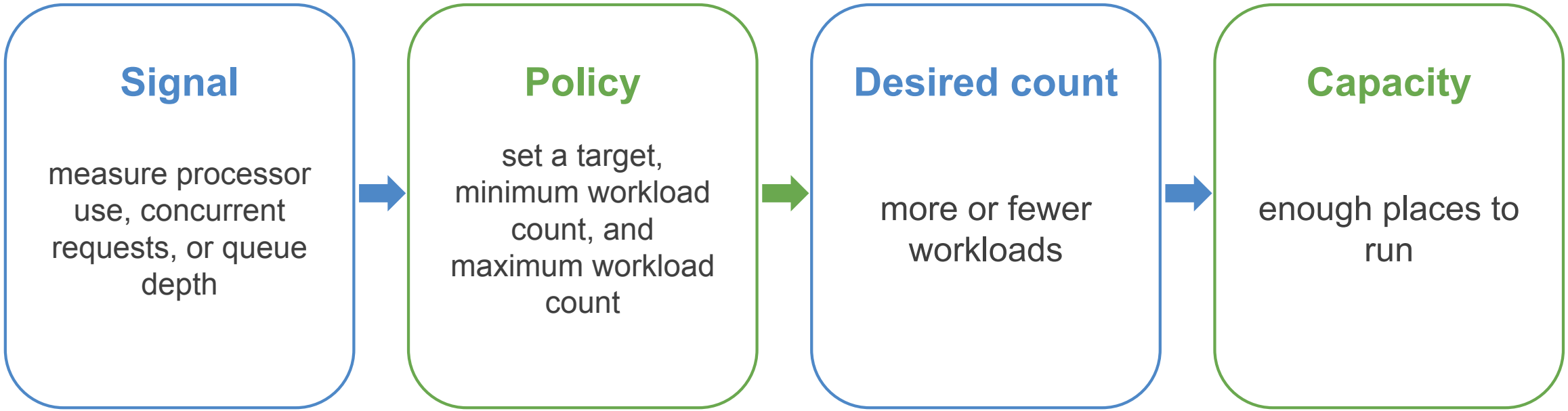
Google routes requests
Google starts and stops instances
Team configures image, ingress, identity + scaling limits

Data and external services remain separate dependencies.

Cloud Run or ECS/Fargate — What You Still Configure

Question	Cloud Run	ECS + Fargate
Deploy unit	One request-serving service	Task definition + service
Scaling	Google adjusts instances within team-set limits	Team sets task count/policy; AWS supplies hosts
Networking	Team sets ingress + network connectors	Team configures VPC, subnets + security
Team configures	image/source, identity, concurrency, min/max	task/service, IAM, health + scaling policy

Scaling — Signal, Policy, Capacity



Downstream limits still bound the result.

Functions — Triggered, Bounded Work

HTTP
Queue
Schedule



handler

one focused result

Longer or multi-step work belongs in a job or workflow.

Functions — Fit the Workload

GOOD FIT



- webhook or event handler
- scheduled cleanup
- small bursty operation

EVALUATE CAREFULLY



- sustained HTTP concurrency
- long-running or heavy startup
- stateful connections

Warm reuse can help latency; it is not durable state.

Functions — One Request or a Shared Instance?

LAMBDA

one request at a time

one request at a time

overlap creates more busy environments

CLOUD RUN FUNCTIONS

several requests may share one instance

configured concurrency changes instance count

Concurrency changes active capacity and downstream pressure.

Functions — Compare Runtime Contracts

Question	AWS Lambda	Cloud Run functions
Concurrency	One concurrent invocation per standard environment	Configurable per instance
Duration	Bounded execution contract	Limits vary by trigger and mode
Packaging	Function package or image	Source built into a provider-operated runtime
Workload identity	Execution role	Service account

Compute Summary — Match Workload to Runtime



FUNCTION

triggered, bounded work



REQUEST-SERVING SERVICE

provider runs routing + instance lifecycle
team configures image + service policy



ORCHESTRATED

team configures tasks/pods, placement + network policy
EC2 or Fargate decides host operations

Compute Contracts — What the Provider Operates

Each service moves a named infrastructure layer—not all responsibility—to the provider.



EC2

AWS: hardware + hypervisor

Team: guest OS + app



ECS/EKS + EC2

AWS: control plane

Team: nodes + workloads



Fargate

AWS: control plane + task hosts

Team: task + network policy



Cloud Run

Google: routing + instances

Team: service config + code



Lambda

AWS: event routing + runtime

Team: function config + code

Provider-operated infrastructure does not settle security, tenancy, data, or recovery.

AWS ↔ Google Cloud — Approximate Neighbors

Workload layer	AWS	Google Cloud
Virtual machine	EC2	Compute Engine
Kubernetes control plane	EKS	Google Kubernetes Engine (GKE)
Container execution	ECS + Fargate	Cloud Run (different contract)
Functions	Lambda	Cloud Run functions
No exact pair	ECS + EC2	Compare requirements

Outline

1. Prerequisites
2. The Cloud Service Spectrum
3. **Beyond Compute: Frontend, Databases & Services**
4. What Should I Use?

These Names Describe Different Jobs

Name	Kind	Job in the system
Python	Programming language	Write backend application logic
JavaScript	Programming language	Run browser behavior; it can also run on servers
FastAPI	Python web framework	Expose HTTP API routes such as POST /predict
React	UI library	Build the browser interface from components
Vite	Frontend build tool	Run development; build production browser files
Next.js	React framework	Build React UI plus optional server behavior
CDN	Delivery network	Cache and deliver content from locations near users
CloudFront	AWS CDN service	Provide AWS's CDN infrastructure

Static Frontend — Two Browser Paths

User browser



CDN

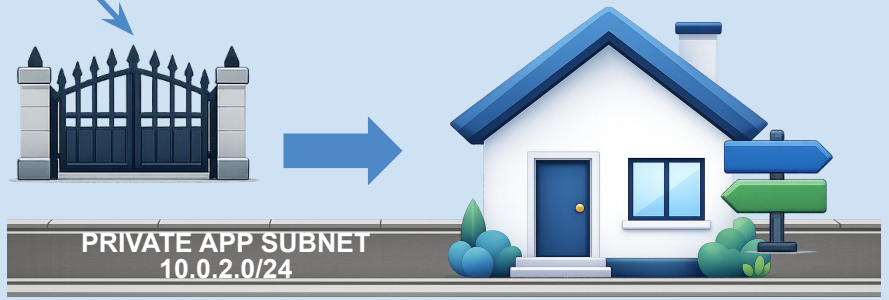
cached copies near users

AWS example: CloudFront

PATH 1 — STATIC APPLICATION FILES

The browser downloads and caches page structure (HTML), styles (CSS), and JavaScript.

VPC — 10.0.0.0/16



PATH 2 — REQUEST TO THE APPLICATION SERVICE

Static Hosting — Who Assembles the Delivery Pipeline?

GIT-CONNECTED HOSTING PLATFORM



- provider runs build/deploy + TLS/CDN
- team supplies repo/artifact + build settings
- team sets domains, redirects, cache + access

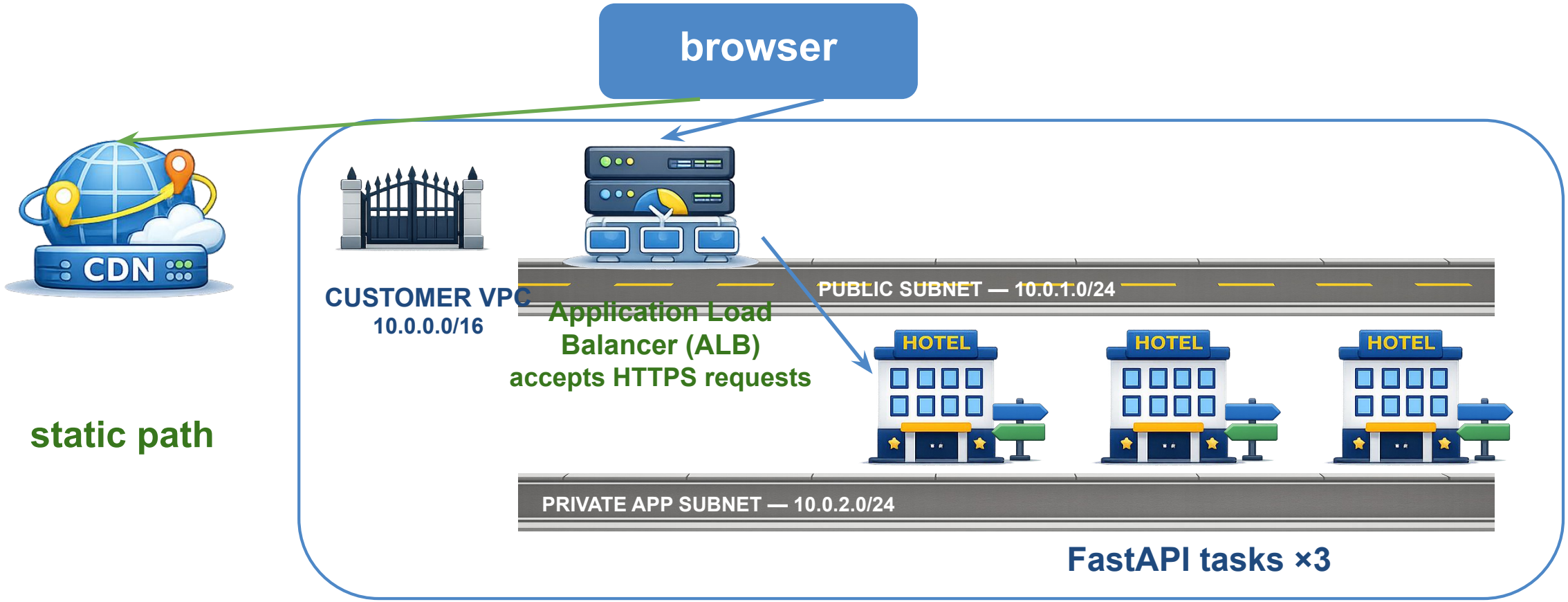
OBJECT STORAGE + EDGE DELIVERY



- providers run storage + edge services
- team builds upload + origin pipeline
- team sets cache, invalidation, TLS + access

Both use provider services; the difference is how much delivery workflow your team assembles.

Frontend + VPC — Two Request Paths



Static delivery stays outside the VPC; application requests enter at the ALB.

Vector Data — Three Deployment Boundaries



CHROMADB ON EC2

team patches VM + ChromaDB
team designs backup + recovery



POSTGRESQL + PGVECTOR SERVICE

provider patches DB infrastructure
team owns schema + restore testing



VECTOR SEARCH SERVICE

provider operates service infrastructure
team owns index, access + recovery settings

Vector Data — Compare the Tradeoffs

Question	ChromaDB on EC2	PostgreSQL service + pgvector	Vector search service
Operations	Team patches VM + ChromaDB	Provider patches DB infrastructure	Provider operates service infrastructure
Data model	Vector store + metadata	Vectors + relational SQL	Service-specific model
Recovery	Team designs backup + tests restore	Team sets retention/HA + tests restore	Contract-specific; verify + test

Benchmark representative queries and failure recovery.

Managed PostgreSQL — What the Provider Actually Operates



application service

SQL over a private path



provider-operated PostgreSQL

Provider operates

database hosts + storage
maintenance + patch workflow
backup + monitoring mechanisms

Team sets network, access, backup/HA—and owns schema, queries, migrations, pooling + restore tests.

PostgreSQL Services — AWS and Google Cloud

AWS



- Amazon Relational Database Service (RDS) for PostgreSQL
- Aurora PostgreSQL-compatible

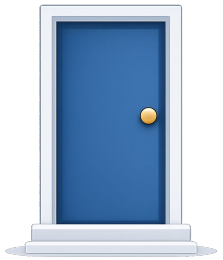
GOOGLE CLOUD



- Cloud SQL for PostgreSQL
- AlloyDB for PostgreSQL

High availability and recovery are configurations—not defaults.

Authentication — Sign In, Issue, Verify



1 User signs in

credentials go to the identity provider



2 Token issued

signed identity claims



3 Backend verifies

checks the signature, trusted issuer, intended audience, and expiration

A valid token proves identity—not permission.

Authorization — Protect the Action

IDENTITY PROVIDER HANDLES



- sign-up and sign-in
- credential protection and multi-factor authentication
- token issuance and lifecycle

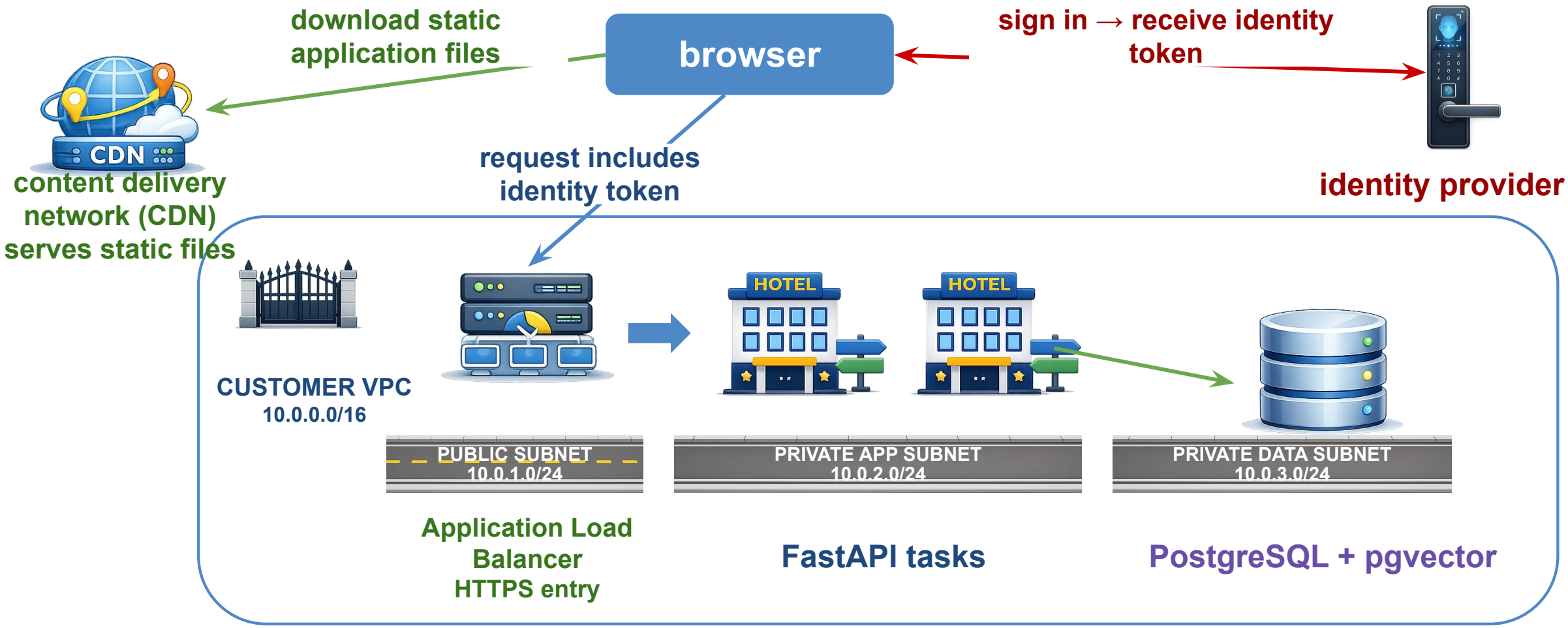
APPLICATION ENFORCES



- validate token claims
- apply role and ownership rules
- permit or reject each action

Authenticated ≠ authorized.

Complete Picture — From Browser to Data



One Availability Zone is shown; production repeats required boundaries across zones.

Next.js — One Codebase, Three Runtime Contracts



STATIC EXPORT

prebuilt files
no request-time server



NODE / CONTAINER

long-lived server
full framework runtime

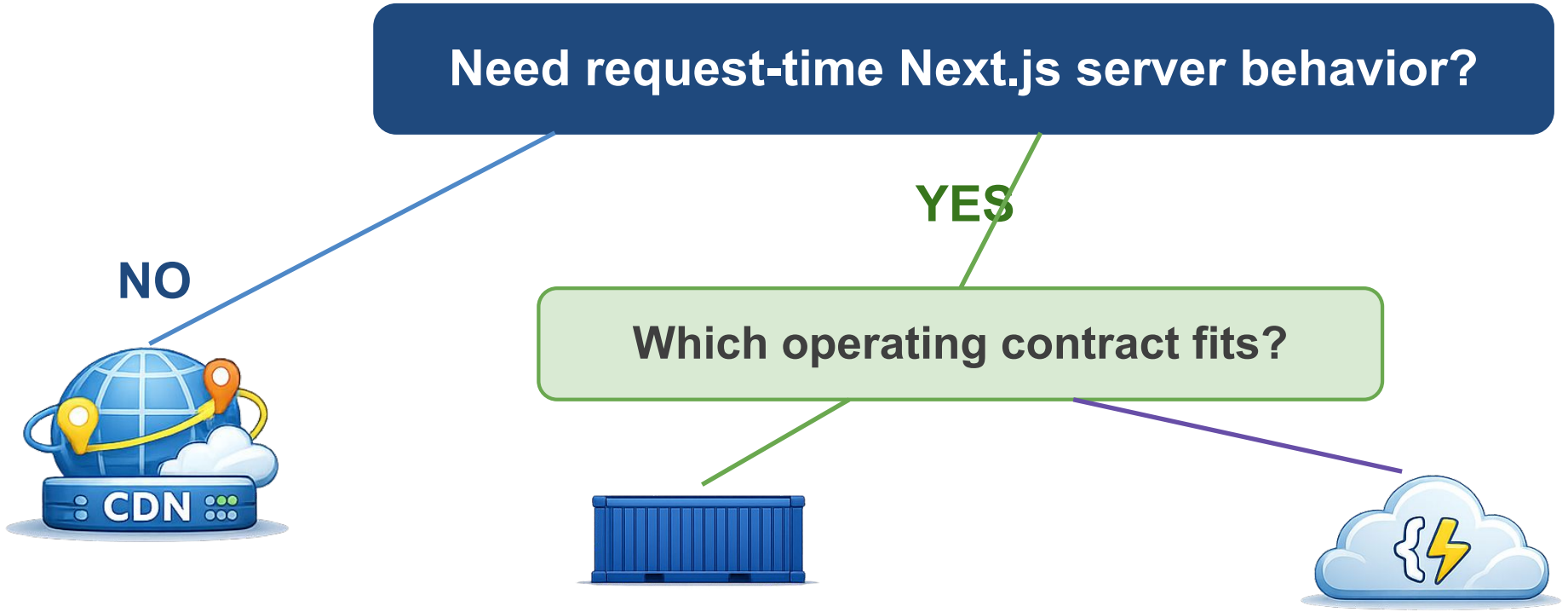


PLATFORM ADAPTER

provider-mapped features
platform limits

One codebase does not define one deployment contract.

Next.js — Choose a Runtime



STATIC EXPORT

NODE / CONTAINER

PLATFORM ADAPTER

Choose the least complex mode that supports required behavior.

Outline

1. Prerequisites
2. The Cloud Service Spectrum
3. Beyond Compute: Frontend, Databases & Services
4. **What Should I Use?**

Step 1 — Draw the Request Path



Frontend

static or server-rendered?



Backend

HTTP, job, or trigger?



Data

relational, vector, or both?



Identity

who and which action?

Start with boundaries and traffic—not service names.

Step 2 — Classify the Data



VECTOR

embeddings + similarity search



RELATIONAL

users + transactions + constraints



BOTH

often PostgreSQL + pgvector

State outlives compute; choose the data contract deliberately.

Step 3 — Know the Stage

DEV / PROTOTYPE



- fast iteration and low cost
- one service + one entry point
- basic logs, alerts, backups

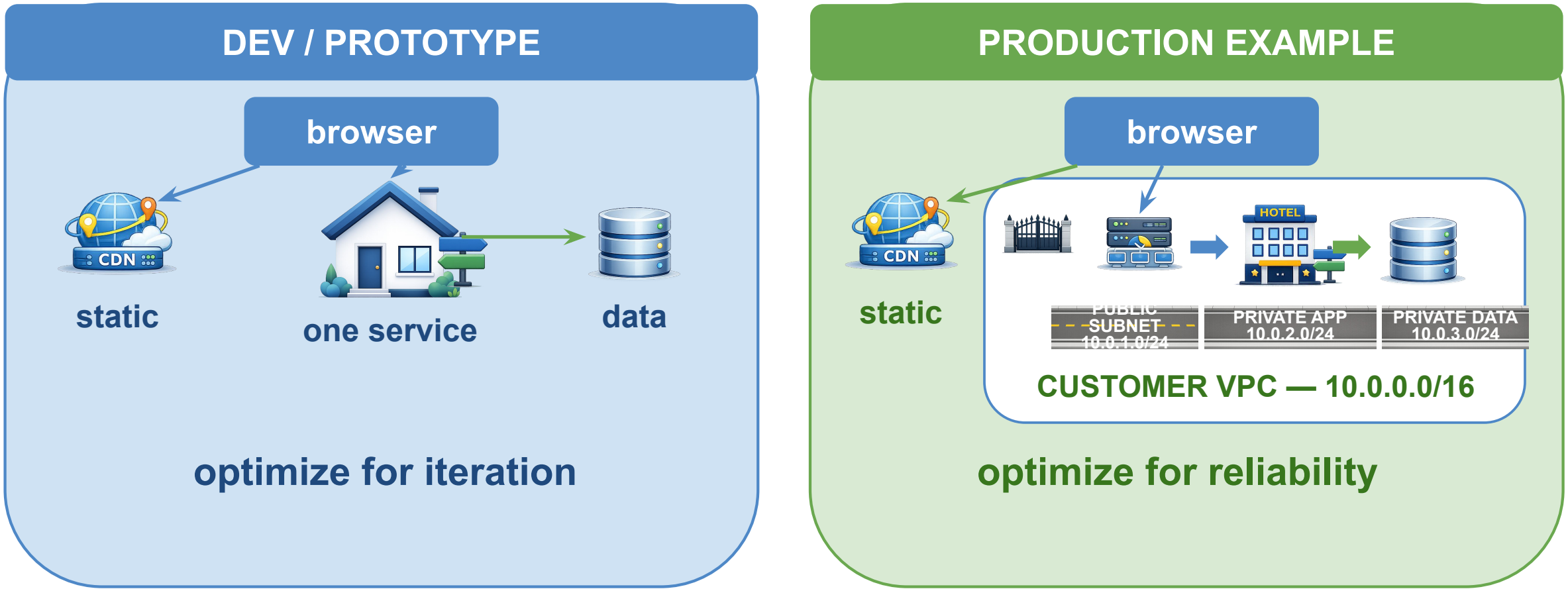
PRODUCTION



- reliability + measured scaling
- private internals; multiple Availability Zones when needed
- monitored backups + tested restore

Security follows users, data, and risk at every stage.

Step 4 — Assemble the Architecture

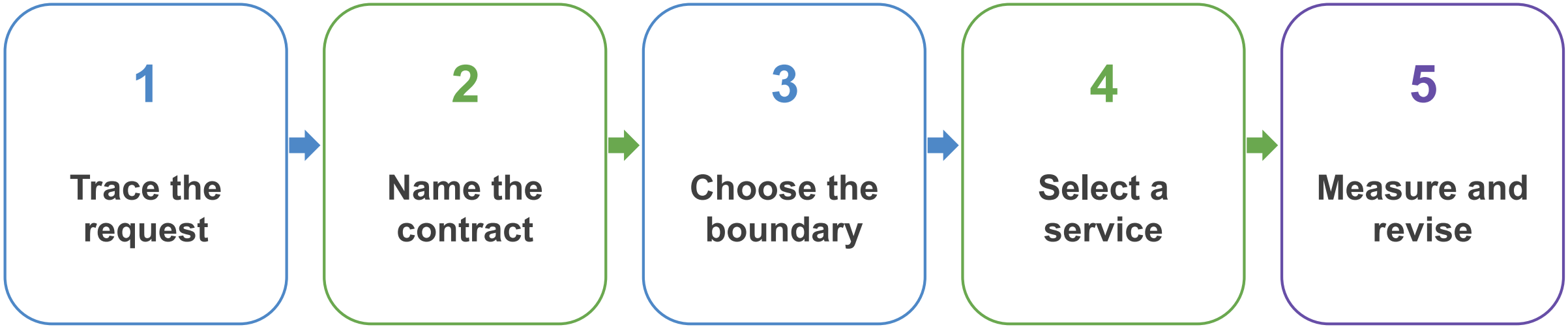


Identity follows users and data risk—not the environment label.

Step 5 — Match Components to Requirements

Component	Requirement	Candidate	Why this contract?
Frontend	_____	_____	_____
Backend	_____	_____	_____
Data	_____	_____	_____
Identity	_____	_____	_____

Quick Reference — Decision Order



Add complexity only when measured needs justify it.



VPC — 10.0.0.0/16

THANK YOU

Sketch one workload’s request path before selecting services.